Universität Rostock



DEBUGGING

Joern Ploennigs · AI4SC

Midjourney: Waves testing a boat, ref. Hokusai

AUSGANGSLAGE

200 Zeilen KI-Code. Falsche Ausgabe.

Debugging-Szenario

Der Code läuft — aber das Ergebnis ist falsch.

- Keine Exception, kein Traceback
- Der KI-generierte Code *sieht* korrekt aus
- Eine Funktion gibt einen Wert zurück — aber den falschen

Wo fängt man an?

Statisches Lesen reicht nicht.

Ausführung muss beobachtet werden.

DIE 5-SCHRITT-KI-DEBUGGING-STRATEGIE

5 SCHRITTE ZUM SYSTEMATISCHEN DEBUGGING VON KI-CODE

Reproduce → Isolate → Hypothesize → Inspect → Fix & Re-verify

Schritt 1 — Reproduzieren

Strategie · Schritt 1 von 5

- Schreibe den **kleinsten Input**, der das Fehlverhalten auslöst
- Entferne alle Abhängigkeiten, die nicht zum Fehler beitragen
- Prüfe: Tritt der Fehler **immer** oder nur unter bestimmten Bedingungen auf?

> **Merksatz**
>
> Ein Bug, den man nicht reproduzieren kann, kann man nicht debuggen.

Schritt 2 — Isolieren

Strategie · Schritt 2 von 5

- Schreibe einen **Unit-Test** direkt für die verdächtige Funktion
- Teste Ein- und Ausgabe isoliert — ohne das gesamte Programm
- Das grenzt das Problem auf eine einzelne Funktion ein

Schritt 3 — Hypothese bilden

Strategie · Schritt 3 von 5

Stelle dir diese Fragen:

- Was *erwartest* du als Ergebnis?
- Was gibt der Code *tatsächlich* zurück?
- Welche Annahme des KI-Codes könnte falsch sein?

> **Merksatz**
>
> Eine gute Hypothese beschreibt die Abweichung zwischen Erwartung und Beobachtung.

Schritt 3 — Typische falsche Annahmen

Strategie · Schritt 3 von 5

- Falscher Datentyp
- Falsche Reihenfolge der Parameter
- Grenzfall nicht behandelt
- Off-by-one im Index

Schritt 4 — Ausführung inspizieren

Strategie · Schritt 4 von 5

- Setze einen **Breakpoint** an der Eintrittsstelle der verdächtigen Funktion
- Führe den Code im Debugger aus — Schritt für Schritt
- Beobachte, **wo** Erwartung und tatsächlicher Zustand auseinanderlaufen

> **Merksatz**
>
> `print()` zeigt ausgewählte Werte. Der Debugger zeigt **alles**.

Schritt 5 — Beheben und re-verifizieren

Strategie · Schritt 5 von 5

- Korrigiere nur die identifizierte Fehlerursache — **minimaler Fix**
- Führe den Unit-Test aus Schritt 2 erneut aus
- Prüfe, ob angrenzende Funktionen ebenfalls betroffen sind

Agentic Workflow

Debugging ist der **Verify**-Schritt auf Ausführungsebene:
prompt → generate → understand → ***verify*** *→ iterate*

Strategie-Übersicht

1. **Reproduzieren**
   Minimalen Fehler-Input finden
2. **Isolieren**
   Unit-Test für die Funktion schreiben
3. **Hypothese**
   Welche Annahme ist falsch?
4. **Inspizieren**
   Debugger, Schritt für Schritt
5. **Fix + Re-Verifikation**
   Testen, ob der Fix vollständig ist

WERKZEUGE IMPLEMENTIEREN DIE STRATEGIE

`print()`-Debugging — Zustände sichtbar machen

Werkzeug · Inspizieren (unterstützt Schritt 1)

```python
def division(zaehler, nenner):
    print(f"Debug: Eingabe Zaehler: {zaehler}")  # Schritt 3: Zustand prüfen
    print(f"Debug: Eingabe Nenner: {nenner}")
    if not isinstance(nenner, (int, float)):
        print(f"Error: Nenner nicht vom Datentyp `int` oder `float`, sondern vom Typ {type(nenner)}")
        raise ValueError(f"Nenner nicht vom Datentyp `int` oder `float`, sondern vom Typ {type(nenner)}")
    elif not isinstance(zaehler, (int, float)):
        print(f"Error: Zaehler nicht vom Datentyp `int` oder `float`, sondern vom Typ {type(zaehler)}")
        raise ValueError(f"Zaehler nicht vom Datentyp `int` oder `float`, sondern vom Typ {type(zaehler)}")
    elif not nenner:
        print("Warning: Division durch 0")
        return None
    ergebnis = zaehler / nenner
    print(f"Info: Ergebnis {zaehler}/{nenner} = {ergebnis}")
    return ergebnis
```

+ Einfach und sofort einsetzbar
- Ausgewählte Werte — nicht der vollständige Zustand
- Viele Ausgaben verdecken den eigentlichen Fehler

`logging` — Gefilterte Sichtbarkeit

Werkzeug · Inspizieren mit Filtern

```
import logging
log = logging.getLogger("meinlog")

log.setLevel(logging.WARNING)  # nur Warnungen und Fehler
log.setLevel(logging.DEBUG)    # volle Sichtbarkeit beim Debuggen
```

Log-Level: `DEBUG` → `INFO` → `WARNING` → `ERROR` → `CRITICAL`

+ Kategorisierte Ausgaben — leichter zu filtern
+ Level dynamisch anpassbar — kein Code-Umbau nötig

○ Erfordert initiale Konfiguration

Grafischer Debugger — Voller Zustandsüberblick

Werkzeug · Inspizieren (Schritt 4)

Breakpoints — Ausführung gezielt unterbrechen:

- An einer bestimmten Zeile anhalten
- Bei einer bestimmten Exception stoppen

Was der Debugger zeigt:

- Aktuelle Zeile (hervorgehoben)
- Alle Variablen im Speicher
- Den gesamten Call Stack



MULTI-FUNKTIONS-MODUL WALKTHROUGH

KI generiert oft mehrere Funktionen

Walkthrough · Schritt 4 — Inspizieren

KI-generierter Code besteht häufig aus **mehreren verschachtelten Funktionen**:

```
def verarbeite_daten(daten):
    bereinigt = bereinige(daten)
    ergebnis  = berechne(bereinigt)
    return formatiere(ergebnis)
```

- `print()` reicht nicht — der Fehler kann in jeder Funktion stecken
- Der Debugger folgt dem Call Stack durch alle Ebenen

Breakpoint setzen → hineinsteigen → Zustand auf jeder Ebene prüfen

Breakpoint am Einstiegspunkt setzen

Walkthrough · Schritt 1 von 3

- Setze den ersten Breakpoint an der **Eintrittsfunktion** (`verarbeite_daten`)
- Rufe die Funktion mit dem **minimalen Fehler-Input** aus Schritt 1 auf
- Die Ausführung hält — du siehst den Eingangszustand vor dem ersten Aufruf

> **Merksatz**
>
> Starte immer am Anfang der Aufrufkette, nicht tief im Code.

Durch Funktionsaufrufe schrittweise gehen

Walkthrough · Schritt 2 von 3

Step Into (Hineinsteigen):

Debugger springt in die aufgerufene Funktion

Step Over (Überspringen):

Debugger führt die Funktion aus, ohne hineinzuspringen

Step Out (Heraussteigen):

Debugger führt die Funktion zu Ende und kehrt zurück

Wann welches nutzen?

- Funktion *verdächtig* → Step Into
- Funktion *bekannt korrekt* → Step Over
- Zu tief geraten → Step Out

Zustand auf jeder Ebene inspizieren

Walkthrough · Schritt 3 von 3

- Prüfe nach **jedem Funktionsaufruf**: Entspricht der Rückgabewert der Erwartung?
- Vergleiche den **Call Stack** — welche Funktion hat was geändert?
- Sobald der Rückgabewert abweicht: **Hypothese bestätigt** → Schritt 5: Fix

> **Agentic Workflow**
>
> Der grafische Debugger ist mächtiger als `print()` für KI-Module mit mehreren Funktionen — er zeigt den vollständigen Zustand ohne Code-Änderungen.

Übungsaufgabe: Fehler systematisch eingrenzen

Praxis · Strategie anwenden

Eine KI liefert für `division("10", 2)` oder `division(10, 0)` unerwartete Ergebnisse.

01 Formuliere einen minimalen Testfall für beide Eingaben.

02 Notiere eine Hypothese, welche Annahme im Code verletzt ist.

03 Entscheide: Reicht `print()`, oder würdest du direkt den Debugger verwenden?

04 Beschreibe, woran du erkennst, dass dein Fix erfolgreich re-verifiziert wurde.

> **Hinweis**
>
> Arbeite die vier Punkte in der Reihenfolge der 5-Schritt-Strategie ab.

QUESTIONS